

A Frontier Model That Behaves Itself

colibrì on LIBR compute — what it is, how you use it, and how fast it really goes

`libr-local-llm` — the next phase

The whole thing in one sentence

A **744-billion-parameter** model, running on one LIBR node, that reads PHI, answers one hard question at a time, and hands the node back when somebody else needs it.

What you gain

- Frontier-scale weights, entirely on-premises
- Reasoning quality no 30B model reaches
- Tool calling, so a real terminal client works
- An API key — the first engine here that has one

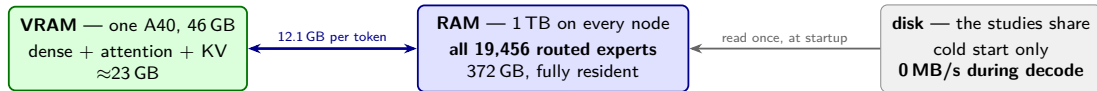
What it costs

- **6–12 tokens per second**
- **27 minutes** to start from cold
- One generation at a time, ever
- About one whole node of the six

Read those two columns together. This is a **consultant**, not an agent loop. One hard question and 80 seconds is a good trade. Forty tool calls is not, and no engineering in this deck changes that.

Why a 744B model runs on a machine that cannot hold it

A 744B mixture-of-experts activates only about **40B parameters per token** — and only the *routed experts* change from one token to the next.



Think of it as a JIT, but for weights. A compiler JIT never compiles the whole program — it watches what actually runs. colibrì makes the same bet: parameters are not state to be held, they are **data to be staged**, exactly when the router proves they are needed.

Placement decides speed and nothing else. The router's decisions and the weights' precision are identical whether an expert answered from VRAM or from disk. You cannot make it *wrong* by placing it badly — only slow.

Where the speed comes from, and where it stops

Every token has to read the experts it routes to. That single fact is the whole ceiling.

$$\begin{aligned} 8 \text{ experts} \times 20.1 \text{ MB} \times 75 \text{ layers} &= 12.1 \text{ GB touched per token} \\ \text{tokens/second} &= \text{memory bandwidth} \div 12.1 \text{ GB} \end{aligned}$$

On the reference machine colibrì was tuned on, the expert path sustained **measured 19.7 GB/s** — only 23% of that host's memory — and the ceiling was **3.0 tok/s**. Everything else in the engine is bounded above by that line.

	bandwidth	ceiling
reference host, as measured	19.7 GB/s	3.0 tok/s
reference host, memory saturated	85 GB/s	13.0 tok/s
our node (2× Ice Lake, 8 channels/socket)	~410 GB/s theoretical	—

The gap between the first two rows is why this is a *tuning* problem rather than a hardware problem, and the third row is why our node is worth tuning.

Our node is the good case, for two reasons

	reference host (4×A6000)	LIBR compute30x
CPU	EPYC Zen 2, 24c/48t, AVX2 only	2× Xeon Ice Lake, 48c/96t, AVX-512 + VNNI
RAM	264 GB DDR4-2400, ~85 GB/s	1 TB DDR4-3200, ~410 GB/s
residency	367 of 429 GB — disk stays in the path	372 of 1024 GB — disk leaves entirely
GPU	4× A6000 48 GB, sm_86	1–4× A40 46 GB, same generation
storage	local NVMe, 2.9 GB/s	NFS, 0.23 GB/s — 12× worse, cold start only

Two rows matter and the rest is detail.

- **AVX-512 VNNI** gives the int4 kernels a path the reference host did not have. colibrì picks it automatically — *if you compile for this machine*. The default build target is AVX2 and would throw it away.
- **1 TB of RAM** holds the entire model. That removes disk from decode permanently, which is the configuration behind every good number colibrì has ever published, and which the reference host could not reach.

The counter-intuitive result: one GPU, not four

Having 1 TB of RAM changes what the GPU is *for*.

What the GPU is for

The **dense and attention** tensors. ≈ 12 GB, plus ≈ 11 GB of KV cache. **23 GB — one A40 holds it.**

What more GPUs would add

More *experts* resident in VRAM instead of RAM. Which, once the model is RAM-resident, is worth. . .

measured $\times 2.8$ ($1.53 \rightarrow 4.26$ tok/s)

measured under 2%

colibri's own controlled A/B: VRAM-heavy (188 GB) and balanced (176 GB) measured **2.81 vs 2.78 tok/s** — indistinguishable. Raising the *RAM* budget by 30 GB bought **+25%**.

So the service never needs compute306. It takes one A40 of the seven on the cluster and leaves the only four-GPU node completely alone. That is the strongest citizenship position available to us, and it falls out of the arithmetic rather than out of good manners.

The number, and where it comes from

colibrì's source carries measurements taken on a **2-socket Ice Lake, 48 cores, model fully resident** — the same CPU generation, core count, and residency condition as ours.

Condition	tok/s	expert-matmul	source
before the VNNI int4 kernel	3.65	67.8 GB/s	colibri.c:541
with AVX-512 VNNI (automatic, if compiled for it)	3.85	89.5 GB/s	colibri.c:541
baseline of the next campaign	4.20	—	colibri.c:554
plus XEXP=1 (one thread team per block)	4.68	131.9 GB/s	colibri.c:554

Those runs are **CPU-side** — none used the GPU for the dense path. Add that $\times 2.8$ lever, partially, and you get:

projected 6–12 tok/s, most likely around 8. A projection built from measured parts: a measured 4.68 tok/s floor on our CPU class, times a partial capture of a measured GPU lever. The first real run on our hardware replaces this line.

XEXP=1 was **neutral or negative on a 24-core box**, which is why it is opt-in. On 48 cores it is the largest CPU-side lever there is.

Four things that do not help, with evidence

The obvious idea	What was measured
More GPUs	Under 2% once the model is RAM-resident. See the previous slide.
More users at once	The engine <i>serialises</i> . 1.32 / 1.51 / 1.46 tok/s aggregate at 1 / 2 / 4 clients — concurrency divides throughput, it does not scale it.
Disk prefetch machinery	Turning it off was worth +26% . Once experts are resident the disk reads 0 MB/s, so prefetch only burns CPU — the scarce resource.
Maximising memory	With the dense path on the CPU, a 176 GB configuration measured <i>slower</i> than a 99 GB one. Gains also do not compose.

And one hidden trap that invalidates naive benchmarking entirely. colibrì keeps a learned routing profile on disk that survives restarts. An *identical* configuration measured **5.46 tok/s** and then **2.56** with no parameter changed. Every measurement must snapshot and restore that file, or it measures nothing.

The real constraint is the 27-minute cold start

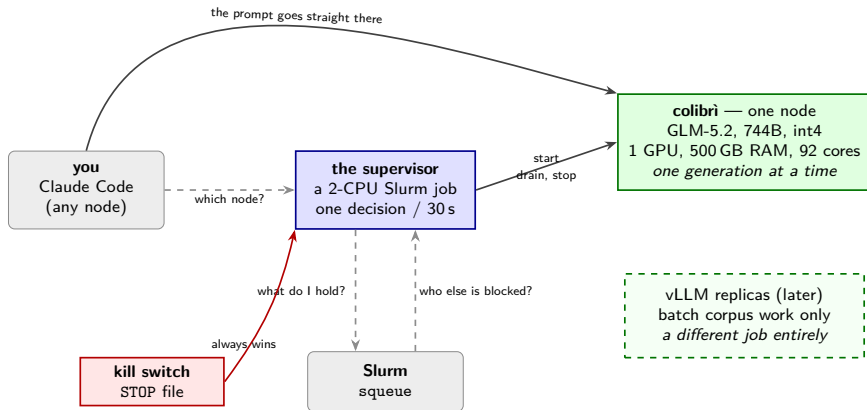
372 GB off the studies share at **measured** 230 MB/s. That number, not the GPU, drives every remaining decision — and it collides with how this cluster's partitions are configured.

Partition	Time limit	Can be frozen?	Verdict
c3_short	9 h	no	27 min of every 9 h is a reload — 5% duty lost, 2.7 restarts a day
c3	7 d	yes, silently	a c3_short job can SIGSTOP it; the client just hangs
c3_accel	7 d	no	the only long- <i>and</i> -safe option — but it is the four-GPU node we just proved we do not need

The inversion worth noticing. Having argued we do not need compute306's *cards*, the strongest reason to run there is its *partition* — the only place offering a 7-day life with no preemption exposure.

The plan says c3_short, reluctantly, and takes the 5%. Holding the only four-GPU node for a service that does not need four GPUs is indefensible. **One test may settle it:** a node has 1 TB of RAM and the model is 372 GB, so a restart on the *same* node may find it still in page cache.

The system is four boxes



The supervisor remembers nothing. Every 30 seconds it asks Slurm what it holds and who is waiting, then takes *one* action. It keeps no state, so a crash loses none — and the backend keeps serving while it is gone.

What you actually type — and it is Claude Code

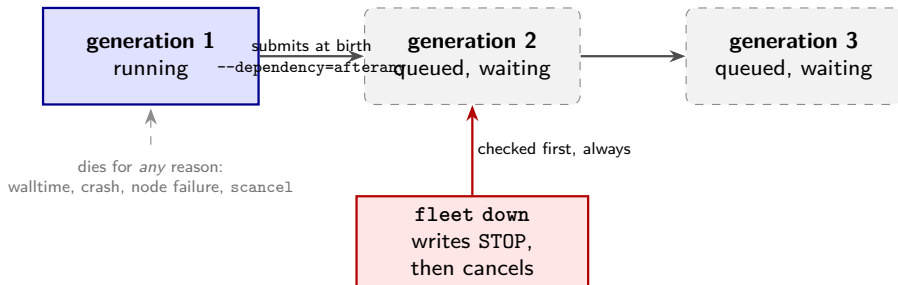
colibrì speaks the **Anthropic Messages API** on the same port as everything else, and GLM-5.2 supports tool calling. So the client is the one you already use — three environment variables, no shim:

```
export ANTHROPIC_BASE_URL=http://127.0.0.1:8000
export ANTHROPIC_API_KEY=your key
export ANTHROPIC_MODEL=glm-5.2-colibrì
claude
```

Command	What happens
<code>fleet code</code>	sets those three, steps onto the backend's node, starts Claude Code
<code>fleet status</code>	what is up, what is warm, what it is waiting on
<code>fleet up / fleet down</code>	start the supervisor / stop everything and stay stopped

And colibrì has an API key, which ollama never did. COLI_API_KEY closes a gap this repo has had recorded as *live* since August: a loopback port is not a boundary against other users on the same node, and until now nothing else was one either.

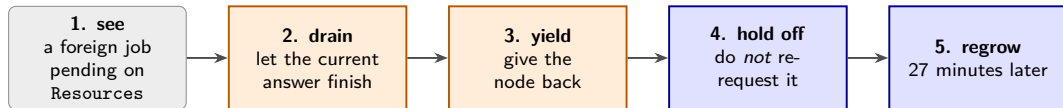
Restarting itself, and the switch that stops it



Each generation submits its successor the moment it starts, with a dependency that fires on *any* termination. A pending job costs nothing, so the queued successor is free insurance.

The kill switch is a file, not a signal. `fleet down` writes `STOP` *before* it cancels anything, so the successor starts, reads it, and exits in two seconds. It works when the supervisor is wedged and when its node is gone — a kill switch that depends on the thing it kills is not one.

Citizenship — and where the pivot costs something



What survives, and it is real

- We hold **one GPU of seven**, never compute306
- One backend. Never two. Ever.
- Idle release still fires — on a 3-hour clock, not 30 minutes

What it honestly costs

- A yield takes the service down for **27 minutes**
- ~92 CPUs and 500 GB billed continuously
- So it will yield *reluctantly*, and rarely

A 27-minute asset released over a lunch break is thrashing, and a fleet that thrashes is worse than a fleet that is simply smaller. If the standing cost is unacceptable, the answer is a shorter walltime — not a cleverer policy.

vLLM is the other half, and not a competitor

colibrì serves **one generation at a time**, at any tuning, forever. So it cannot do corpus work — and the two engines stop overlapping entirely.

	colibrì	vLLM
workload	one hard question, interactively	ten thousand documents, unattended
model	GLM-5.2 744B int4	MedGemma 27B, 8-bit
shape	one long-lived backend, single slot	N single-GPU replicas, batched
plane	HTTP on loopback	filesystem queue — no socket at all
for	frontier reasoning on PHI	PSYCH-ASR Stage 3c, schema-guided JSON

Do not build a router between these two. They share no caller and no workload. The arbitration layer the original design was going to need has been deleted, which is the pivot's one unambiguous simplification.

The batch queue is also the strongest PHI control in the design, because it removes the socket entirely: a worker killed mid-item costs one item, not the pass.

Which model, inside colibrì

Eight families run on the same engine. The filter is brutal: it must fit in 1 TB of RAM, **and** support tool calling, or a terminal client cannot drive it.

Model	total / active	container	tools	verdict
GLM-5.2	744B / 40B	372 GB	yes	the default — every number in this deck belongs to it
DeepSeek V4 Flash	284B / 13B	167 GB	yes	measure it. One third the active parameters could mean real speed
GLM-5.3-Flash	321B / 40B	195 GB	yes	a lighter GLM; needs conversion
Inkling	975B / 41B	469 GB	no	unusable — rejects tool calls with HTTP 400
Kimi K3	2.8T / 104B	1.6 TB	yes	does not fit in RAM. Dead here

Quality is not free, and the number is known. GLM-5.2's int4 container measures —**8.2 percentage points** against the unquantized model, concentrated on the hardest questions.

Whether 744B at int4 beats a well-served smaller model at 8-bit *for our tasks* is a blind side-by-side nobody has run. The pivot makes that comparison more important, not less.

Build order — and you can stop early

Phase	What lands	Exit criterion
P0	ten verification tests	a real tok/s number on our hardware , under the snapshot protocol
P1	one backend, tuned, run by hand	Claude Code finishes a real task against it, with a tool call
P2	the sbatch, <code>fleet up</code> / <code>status</code> / <code>down</code>	one command, and it refuses to claim success before a token exists
P3	supervisor, chain, kill switch	survives two rollovers; <code>fleet down</code> makes it stay down
P4	idle release, caps, yield, hold-off	yields to a real blocked job, does not take the node back early
P5	vLLM + the batch queue	a corpus pass survives a worker being killed

Stop after P1 and you already have what was asked for, run by hand. P2–P4 are what make it survive being left alone. **P0 comes first and is not optional** — the previous plan could build its control plane first because ollama was already proven. Nothing here is.

Five decisions that are yours, not mine

Question	Proposed	Cost if changed
Partition: 9-hour and safe, or 7-day on the four-GPU node?	c3_short, reluctantly	5% duty lost, versus holding the scarcest node
Is ~92 CPUs and 500 GB on one of six nodes, continuously, acceptable?	it has to be	if not, the service does not exist in this form
GLM-5.2 only, or measure DeepSeek V4 Flash first?	measure both	13B active vs 40B is a large potential speed difference
Claude Code or opencode as the client?	Claude Code	colibrì speaks Anthropic natively; opencode stays as fallback
Is 6–12 tok/s acceptable for the work you have in mind?	as a consultant, yes	as an agent loop, no , and nothing here changes that

Recorded as open in FLEET-BUILD.md §13. Answer them once, with a date, and the implementation stops guessing.

What “done” looks like

You type fleet code and Claude Code opens against a **744-billion-parameter model running on LIBR hardware**, which may read PHI, because nothing leaves the building.

You ask it one hard question. It thinks for eighty seconds and answers well.

Overnight the fleet releases the node. In the morning you pay 27 minutes, once, and carry on.

If a colleague's GPU job goes pending, the fleet finishes your answer and gives the node back — and the log says so, with their job id, when anybody asks.

Nothing in that paragraph needs an administrator, a login-node daemon, or a person watching.

It is frontier-scale. It is on-premises. It is eight tokens a second.

Two of those three are the reason to build it. The third is why it is a consultant.